COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	10/6/2026	**DATE SUBMITTED:**	10/6/2026
NAME:	Jose Martin C. Rodulfa Dominic Rodnee Panti	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11.1 **Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

ILO A:

Assess if two arrays have any common values.

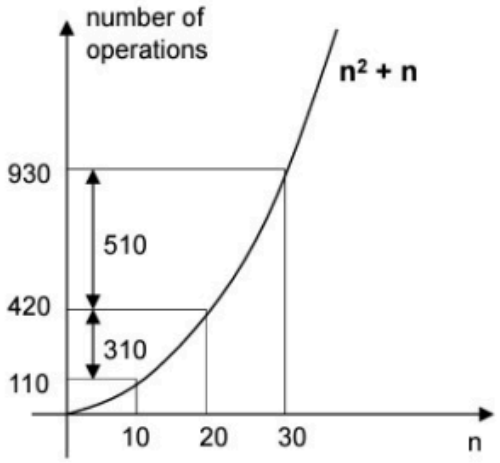
```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Total number of comparisons in the worst case would be: m*n

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

ANALYSIS:

When the arrays are equal, the number of operations increases quadratically as n is squared. The inner loop provides the n^2 while the outer loop adds an n, matching the graph's function.This produces a non-linear parabolic curve simulating that it runs on an $O(n^2)$ time complexity. This results in steep scalability of the number of operations, leading to an inefficiency in larger datasets.

ILO B:

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	Joma: 0 microseconds Rodnee:0 microseconds	Joma: Time taken to search = 0 microseconds Student (351 256) needs vaccination. Rodnee: Time taken to search = 0 microseconds Student (801 406) needs vaccination.	There is no difference in the run-time.
10,000	Joma: 0 microseconds Rodnee: 0 microseconds	Joma: Time taken to search = 0 microseconds Student (351 256) needs vaccination. Rodnee: Time taken to search = 0 microseconds Student (801 406) needs vaccination.	There is no difference in the run-time.
100,000	Joma: 0 microseconds Rodnee: 1 microsecond	Joma: Time taken to search = 0 microseconds Student (351 256) needs vaccination. Rodnee: Time taken to search = 1 microseconds Student (801 406) needs vaccination.	At the 10,000 N mark, one of our devices slightly lag, causing a 1 microsecond difference.

Analysis:

When running the program, the execution speed stayed the same in the 1000 and 10,000 N mark. But in the 100,000, there is a slight lag in one of the hardware devices, concluding that the larger the data set, the more the hardware of a system has to process the code and the longer it is to output results.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

RODULFA:

- ***Theoretical Analysis***: T(n) = n^2 + n;
- ***Experimental Analysis:***

```cpp
prob1RODULF.cpp > ...
#include <iostream>

bool Unique(const int A[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n-1; j++) {
            if (A[i] == A[j]) {
                return false;
            }
        }
    }
    return true;
}

```

```
N = 1000        Operations = 498501     Time = 615300 ms
N = 2000        Operations = 1997001    Time = 2.6211e+06 ms
N = 4000        Operations = 7994001    Time = 8.6343e+06 ms
N = 8000        Operations = 31988001   Time = 4.48734e+08 ms
N = 16000       Operations = 127976001  Time = 2.8037e+08 ms
```

- ***Comparison:*** By comparing the code and theoretical analysis, we can confirm the best and worst case scenario of O(1) and O(n^2) respectively due to the familiar enhanced bubble sort-style implementation.

PANTI:

- ***Theoretical Analysis:***
  - T(n)=n² + n
- ***Experimental Analysis:***

```cpp
#include <iostream>
#include <chrono>


bool Unique(const int A[], int n, long long& op_count) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n-1; j++) {
            op_count ++;
            if (A[i] == A[j]) {
                return false;
            }
        }
    }
    return true;
}
```

```
N = 1000        Operations = 498501     Time = 461100 ms
N = 2000        Operations = 1997001    Time = 1.806e+06 ms
N = 4000        Operations = 7994001    Time = 7.223e+06 ms
N = 8000        Operations = 31988001   Time = 2.88523e+07 ms
N = 16000       Operations = 127976001  Time = 1.17352e+08 ms
```

- ***Comparison:***
  The results match perfectly with the theoretical analysis because the code processes the data set twice, increasing the size of the array results in the total running time becoming four times greater. This clear behaviour proves that the code scales in the time complexity ofO(n²).

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

RODULFA:

- ***Theoretical Analysis:*** **LINEAR =** T(n) = O(n) ; **BINARY =** T(n) =O(log n)
- ***Experimental Analysis:***

```cpp
#include <iostream>

// Naive Recursive Exponentiation: O(n) time, O(n) stack space
long long rpower(int x, int n) {
    if (n == 0) {
        return 1;
    }
    return x * rpower(x, n - 1);
}

// Binary Exponentiation: O(log n) time, O(log n) stack space
long long brpower(int x, int n) {
    if (n == 0) {
        return 1;
    }

    if (n % 2 != 0) { // n is odd
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {         // n is even
        long long y = brpower(x, n / 2);
        return y * y;
    }
}
```

Exponent (n)	rpower Time (ns)	brpower Time (ns)
100	800	100
1000	28800	100
5000	124900	200
10000	187100	100
20000	390800	100

LINEAR = T(n) = O(n) ; Best Case: O(n) ; Worst Case: O(n)
BINARY = T(n) = O(log n) ; Best Case: O(log n) ; Worst Case: O (log n)

- ***Comparison:*** The theoretical analysis assesses the bytes that the program occupies during runtime, while the Experimental Analysis goes more in depth to the space and time complexities of each function. Here, the linear function makes 10 calls, while the binary function has to make a full 20 (hypothetically speaking).

PANTI:

- ***Theoretical Analysis:***
  - Linear: T(n) = O(n)
  - Binary: T(n) = O(log n)
- ***Experimental Analysis:***

```cpp
#include <iostream>
#include <chrono>
#include <iomanip>


// Naive Recursive Exponentiation: O(n) time, O(n) stack space
long long rpower(int x, int n) {
    if (n == 0) {
        return 1;
    }
    return x * rpower(x, n - 1);
}


// Binary Exponentiation: O(log n) time, O(log n) stack space
long long brpower(int x, int n) {
    if (n == 0) {
        return 1;
    }

    if (n % 2 != 0) { // n is odd
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {          // n is even
        long long y = brpower(x, n / 2);
        return y * y;
    }
}
```

```
--- Experimental Analysis ---
Exponent (n)   rpower Time (ns)         brpower Time (ns)
-----------------------------------------------------------------
100            1600                     100
1000           62700                    100
5000           248200                   200
10000          329400                   200
20000          718300                   200
```

- ***Comparison:***
  Even though they yielded the same result, their structural scaling properties are completely different. With rpower, the amount of processing it requires increases linearly as the exponent is increased. In the case of brpower, highly optimal scaling is shown since when the value of n^2, its workload rises by only one constant step. This illustrates the great efficiency advantages that logarithmic algorithms have over linear functions when dealing with a large integer range.

CONCLUSION (Rodulfa):
In this HOA, I was able to try basic algorithm analysis by observing theoretical and experimental runtimes. I was able to observe how quadratic algorithms, O(n^2), quickly become inefficient as input sizes grow, while logarithmic algorithms, O(log n), maintain rapid execution times. Through benchmarking these different approaches, I appreciated seeing how mathematical complexities directly impact real-world program performance

CONCLUSION (Panti):
I was able to see the scaling principles in the code as actual differences in runtime displayed on my terminal window. In the future, I plan to concentrate on improving my method for calculating recurrence relations in the case of complex recursive logic and on sharpening my ability to identify hidden best-case paths within large codebases.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: N/A

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: N/A

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: N/A

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.